

MAP ASSIGNMENT-3

I. Event Validation using isNotNull, isEmail, isNumber

Project Name: ValidationRules

Objective:

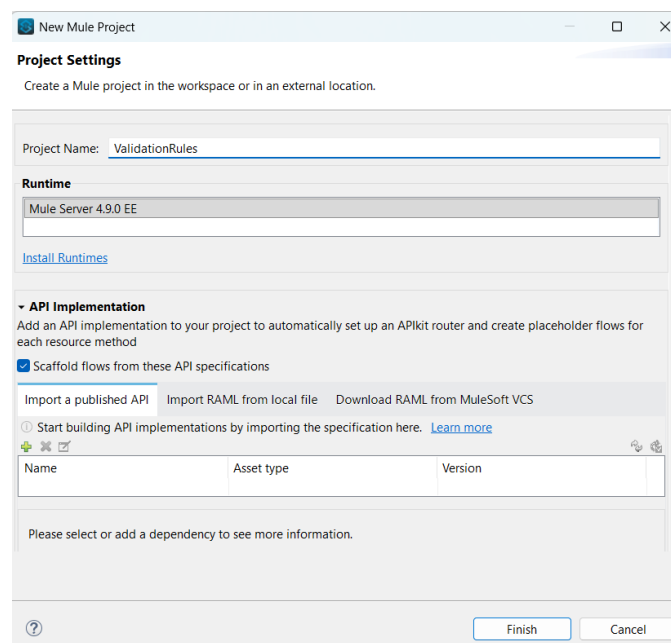
The goal is to validate the incoming request payload in a specific sequence:

1. **Check if Not Null** → payload.name
2. **Check for Valid Email** → payload.email
3. **Check for Number within Range (0–100)** → payload.marks

Finally, the validated payload should be returned using a **Transform Message** component.

Step 1: Create a New Project

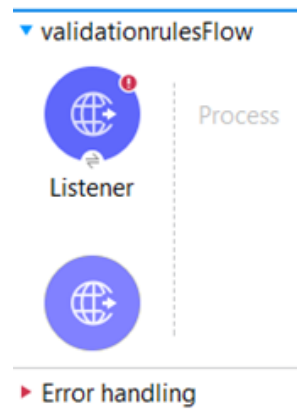
1. Open **Anypoint Studio**.
2. Navigate to **File → New → Mule Project**.
3. Name the project as **ValidationRules**.



Step 2: Configure HTTP Listener

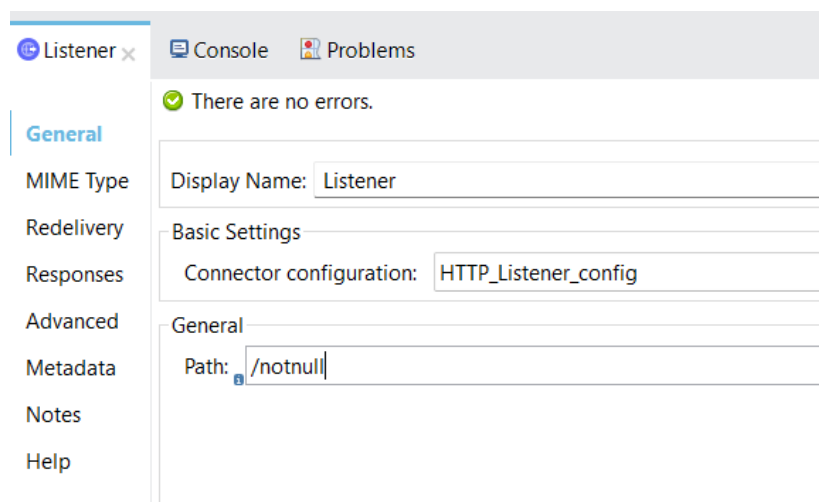
1. Drag an **HTTP Listener** component into the flow.
2. Create a new configuration with the following details:

- **Host:** 0.0.0.0
- **Port:** 8081



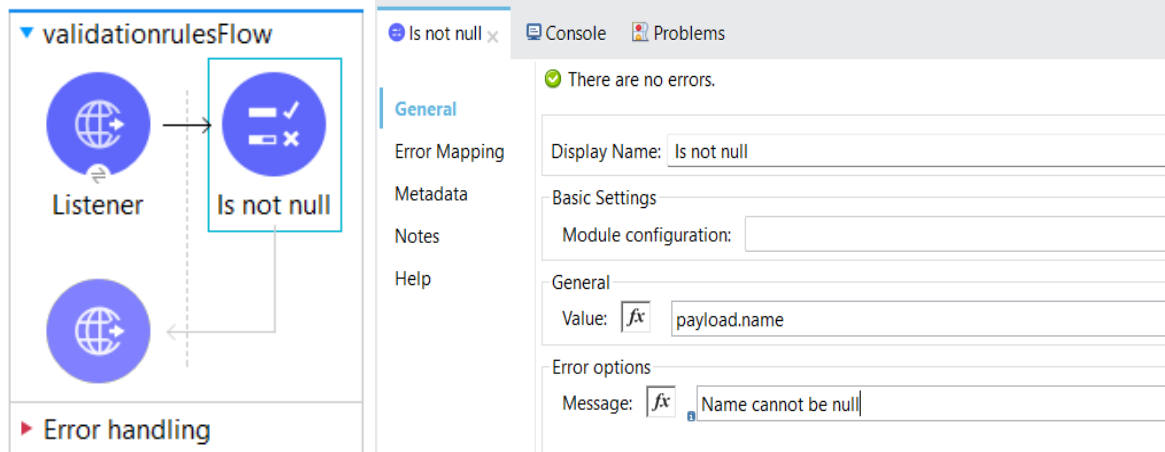
3. In the **Listener** properties:

- **Path:** /validation
- **Purpose:** This endpoint will accept incoming **POST** requests.



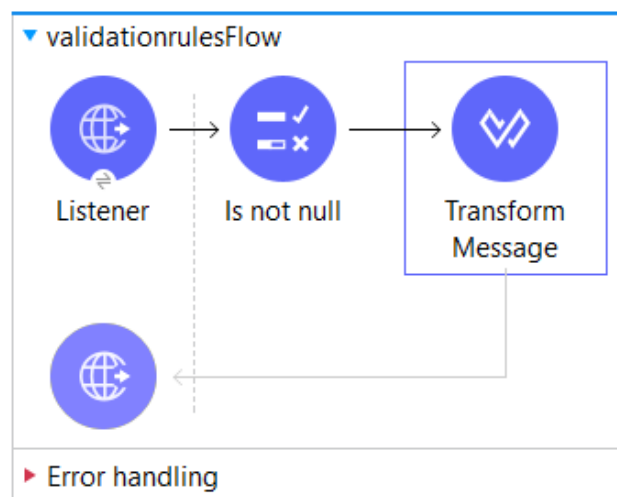
Step 3: Add “Is Not Null” Validation

1. Drag a **Validation** → **Is Not Null** component after the listener.
2. Configure it as follows:
 - **Value:** #[payload.name]
 - **Message:** “Name should not be null.”
3. **Purpose:** Ensures the request includes a non-empty name value.



Step 4: Add Transform Message

1. Add a **Transform Message** component at the end of the flow.
2. Insert the required **DataWeave (DWL)** script to return the payload.



Output Payload

```
1 %dw 2.0
2 output application/json
3 ---
4 payload
```

Step 5: Repeat HTTP Configuration (if needed)

The screenshot shows the MuleSoft IDE interface. On the left, a canvas titled 'validationrulesFlow1' contains a 'Listener' component (represented by a globe icon) and an 'Error handling' section. The 'Listener' component is highlighted with a blue border. On the right, the 'Listener' configuration panel is open, showing the 'General' tab. The 'Display Name' is 'Listener', the 'Connector configuration' is 'HTTP_Listener_config', and the 'Path' is '/email'. The 'Console' tab shows a green checkmark and the message 'There are no errors.'

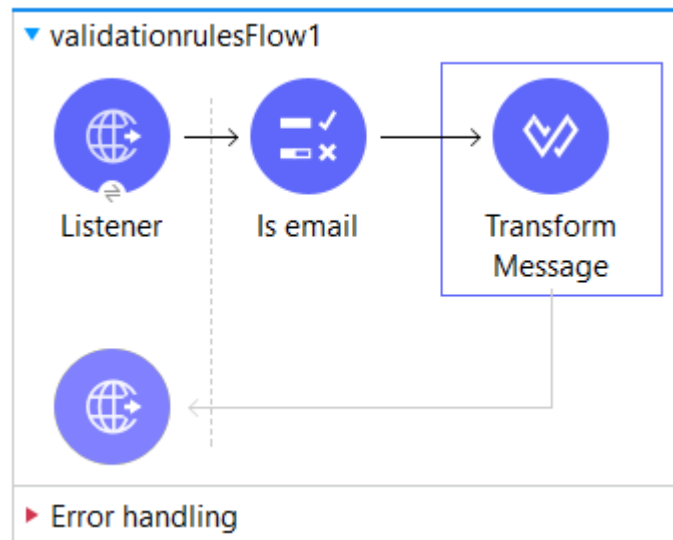
Step 6: Add “Is Email” Validation

1. Add a **Validation** → **Is Email** component.
2. Configure:
 - **Value:** #[payload.email]
 - **Message:** “Please enter a valid email address.”
3. **Purpose:** Validates that the email format is correct.

The screenshot shows the MuleSoft IDE interface. On the left, a canvas titled 'validationrulesFlow1' contains a 'Listener' component (represented by a globe icon) and an 'Is email' validation component (represented by a blue circle with a checkmark and an 'x'). The 'Is email' component is highlighted with a blue border. On the right, the 'Is email' configuration panel is open, showing the 'General' tab. The 'Display Name' is 'Is email', the 'Connector configuration' is 'HTTP_Listener_config', and the 'Path' is '/email'. The 'Console' tab shows a green checkmark and the message 'There are no errors.'

Step 7: Add Transform Message

1. Again, add a **Transform Message** component to display the result.



```
Output Payload ▼ ⚙️ ✎ 🗑️
1 @dw 2.0
2 output application/json
3 ---
4 payload
```

Step 8: Repeat HTTP Configuration (if needed)

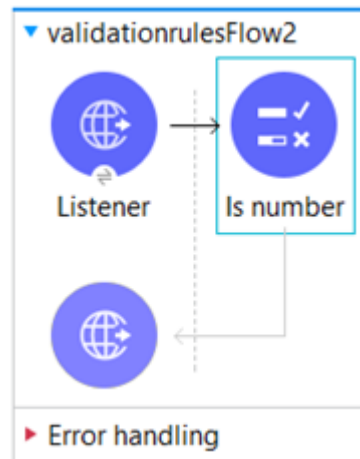
The screenshot shows the IDE interface. On the left, a flow named 'validationrulesFlow2' is visible, containing a 'Listener' node. On the right, the configuration panel for the 'Listener' node is open. The 'General' tab is selected, showing the following settings:

- Display Name: Listener
- Basic Settings: Connector configuration: HTTP_Listener_config
- General: Path: /number

The 'Console' tab shows a message: 'There are no errors.'

Step 9: Add "Is Number" Validation

1. Drag **Validation** → **Is Number** into the flow.



2. Configure:

- **Value:** #[payload.marks]
- **Message:** "Please enter valid marks."
- **Minimum Value:** 0
- **Maximum Value:** 100

3. **Purpose:** Confirms that marks are numeric and fall within the range 0–100.

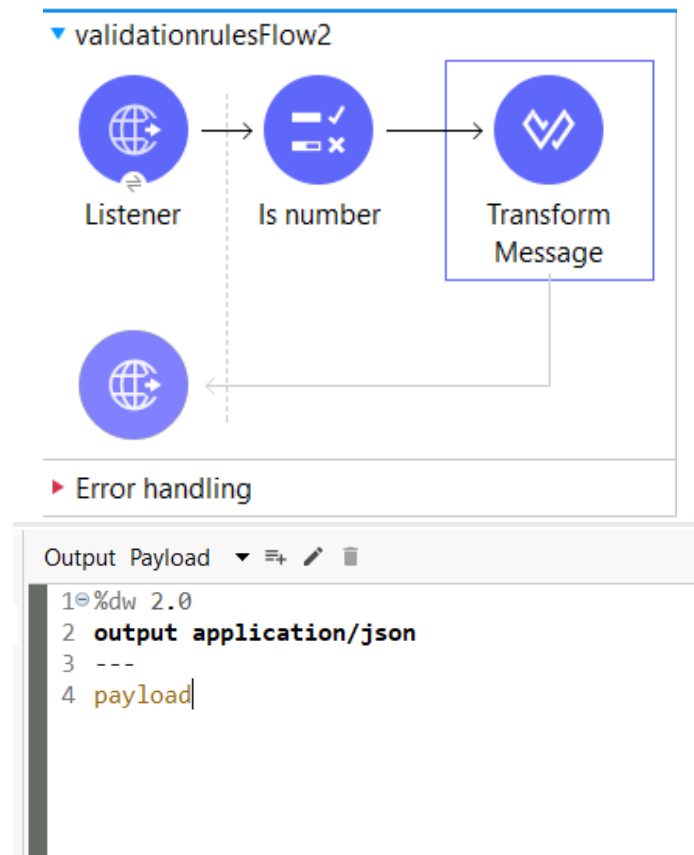
The screenshot shows the configuration window for the 'Is number' validation rule. The window has a sidebar with tabs: General, Error Mapping, Metadata, Notes, and Help. The 'General' tab is selected. The main area shows the following configuration:

- Value:
- Locale:
- Pattern:
- Min value:
- Max value:
- Number type:

Below the main configuration, there is an 'Error options' section with a 'Message' field.

Step 10: Add Transform Message

1. Add the final **Transform Message** to output the validated data.



Output:

HTTP http://localhost:8081/validation

GET http://localhost:8081/validation

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON ▼

```
1 {
2   "name": null,
3   "marks": 18,
4   "email": "patro@gmail.com"
5 }
```

Body Cookies Headers (4) Test Results

Pretty Raw Preview Visualize Text ▼

```
1 the name should be not null
```

HTTP **http://localhost:8081/validation**

GET **http://localhost:8081/validation**

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary **JSON** ▾

```
1 {  
2   "name": "sai",  
3   "marks": 18,  
4   "email": "patrogmail.com"  
5 }
```

Body Cookies Headers (4) Test Results

Pretty Raw Preview Visualize Text ▾ ↺

```
1 enter a valid e mail
```

HTTP **http://localhost:8081/validation**

GET **http://localhost:8081/validation**

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary **JSON** ▾

```
1 {  
2   "name": "sai",  
3   "marks": 188,  
4   "email": "patro@gmail.com"  
5 }
```

Body Cookies Headers (4) Test Results

Pretty Raw Preview Visualize Text ▾ ↺

```
1 enter marks between 0-100
```


II. Choice Router

Project Name: StudentGrades

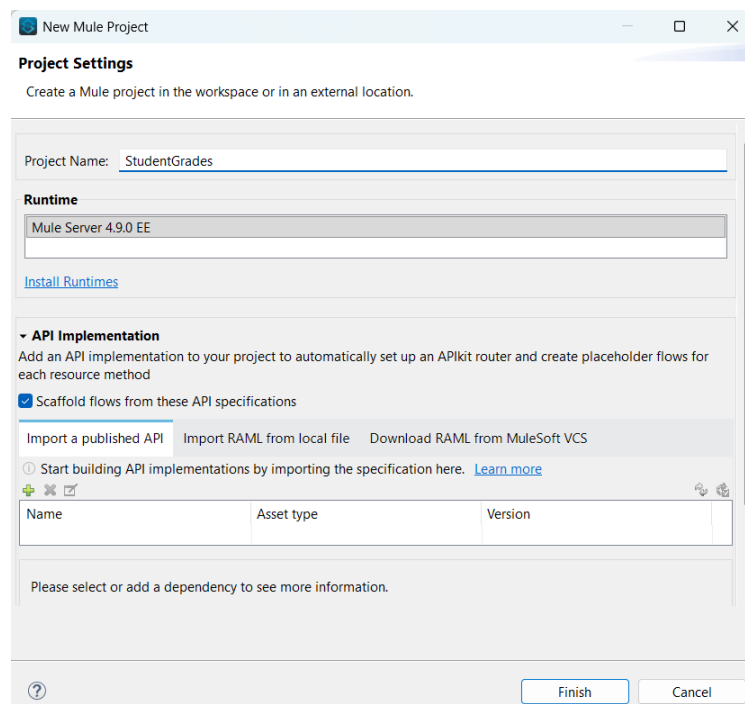
Objective:

Accept a student's marks from the request payload, store them as a variable, and use a **Choice Router** to determine the corresponding grade:

Marks Range	Grade
≥ 90	A
≥ 80	B
≥ 70	C
≥ 60	D
≥ 50	E
≥ 40	Pass
< 40	Fail

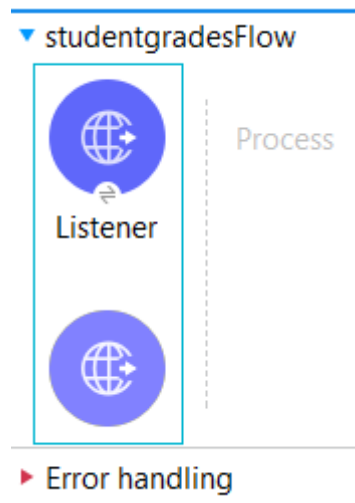
Step 1: Create a New Project

1. Open **Anypoint Studio**.
2. Go to **File → New → Mule Project**.
3. Set the project name as **StudentGrades**.

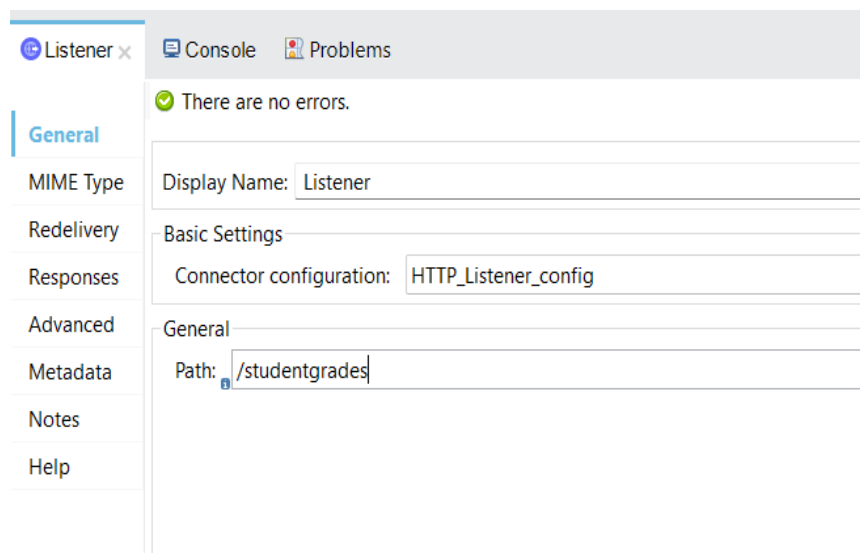


Step 2: Add HTTP Listener

1. Drag an **HTTP Listener** into the flow.

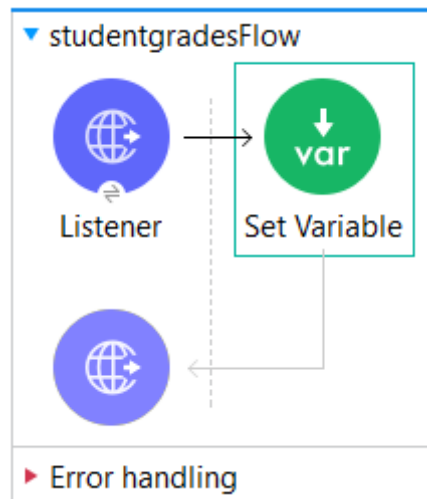


2. Configure the listener as follows:
 - **Path:** /grades
 - **Port:** 8081
3. **Purpose:** To receive incoming requests containing student marks.



Step 3: Set Variable

1. Add a **Set Variable** component after the listener.



2. Configure:

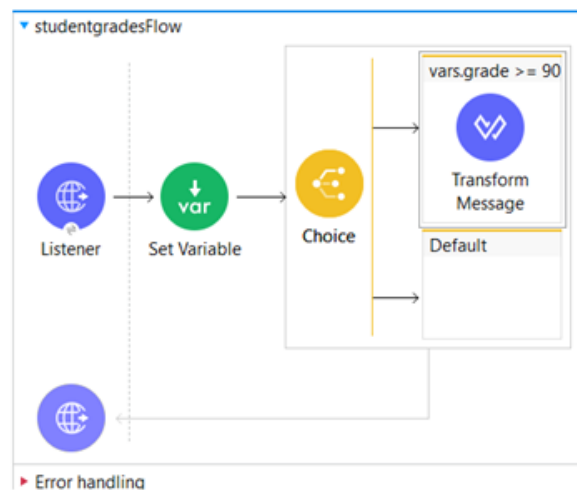
- **Name:** grade
- **Value:** #[payload.grade]

3. **Purpose:** To store the marks from the incoming request into a variable for further processing.

The screenshot shows the configuration interface for the 'Set Variable' component. The 'General' tab is selected. The 'Display Name' is 'Set Variable'. Under 'Settings', the 'Name' is 'grade' and the 'Value' is set to the expression '#[payload.grade]' using the 'fx' (function) button.

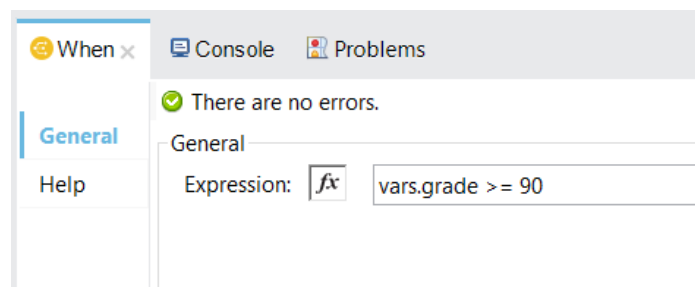
Step 4: Configure Choice Router

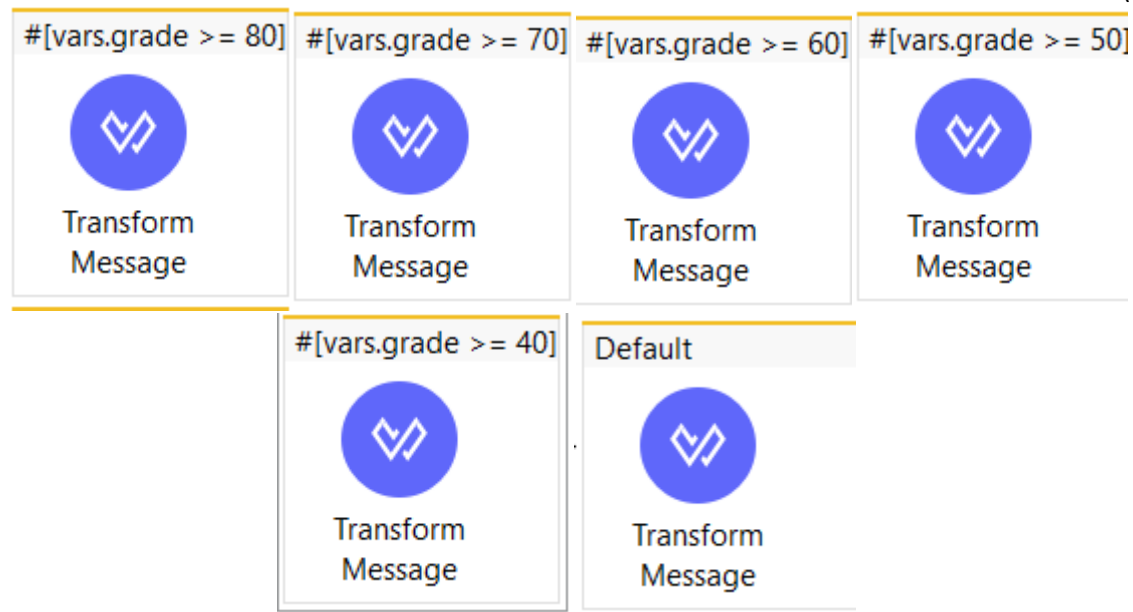
1. Add a **Choice** component after the variable.



2. Define multiple **When** conditions and a **Default** case using **DW (DataWeave)** expressions:

- `#[vars.grade >= 90] → Transform → "Grade: A"`
- `#[vars.grade >= 80] → Transform → "Grade: B"`
- `#[vars.grade >= 70] → Transform → "Grade: C"`
- `#[vars.grade >= 60] → Transform → "Grade: D"`
- `#[vars.grade >= 50] → Transform → "Grade: E"`
- `#[vars.grade >= 40] → Transform → "Pass"`
- **Default:** → Transform → "Fail"





Step 5: Add Transform Message for Each Condition

For each branch, add a **Transform Message** component that returns the respective grade (for example, "Grade: A", "Grade: B", etc.).

Step 6: Run and Test

1. Run the **Mule Application**.
2. Test using **Postman** by sending requests to verify each condition.

